Smart Contract Security Audit Report

1. Project Overview

The SlowMist Security Team received the USDONLINE team's application for smart contract security audit of the USDONLINE on Apr. 10, 2026. The following are the details and results of this smart contract security audit:

Project name :
USDONLINE

The Contract address :

0x7cbab1b8dfd2d7e571846f73f0cc11386b774d59

Link address :

https://bscscan.com/address/0x7cbab1b8dfd2d7e571846f73f0cc11386b774d59

Contract Details

Contract Name	LockStakingV3
Compiler	Solidity ^0.8.20
License	MIT
Bytecode Size	11,976 bytes (within EIP-170 24KB limit)
Primary Type	Fixed-term ERC20 staking with referral hooks
Dependencies	IERC20, IReferralRegistry (optional), IRewardsVault (optional)

2. Audit Items and Results

The following table summarizes the audit checklist applied to this contract. Other unknown security vulnerabilities are not included in the audit responsibility scope.

No.	Audit Item	Subclass	Result
1	Overflow Audit	—	Passed
2	Race Conditions Audit	Reentrancy guard	Passed
3	Authority Control Audit	Permission vulnerability	Passed
		Excessive authority	Passed

No.	Audit Item	Subclass	Result
4	Safety Design Audit	Compiler version	Passed
		Hard-coded address	Passed
		Function return value	Passed
		Call function security	Passed
		Safe ERC20 usage	Passed
5	Denial of Service Audit	External call isolation	Passed
6	Gas Optimization Audit	—	Passed
7	Design Logic Audit	—	Passed
8	"False Deposit" Audit	Fee-on-transfer handling	Passed
9	Malicious Event Log Audit	—	Passed
10	Scoping and Declarations	—	Passed
11	Replay Attack Audit	—	N/A (no signatures)
12	Uninitialized Storage Pointers	—	Passed
13	Arithmetic Accuracy Deviation	—	Passed

Audit Result: Passed

Audit Number: 0X002604240001

Audit Date: Apr 10, 2026

Summary: LockStakingV3 demonstrates materially improved security over its predecessor. All high-severity and critical-severity findings from the v2 review have been addressed. The contract implements defensive patterns including SafeERC20 transfers, two-step ownership transfer, internal principal accounting, rate-limited and lifetime-capped emergency withdrawals, and

try/catch-wrapped external calls to optional referral modules. The comprehensive evaluation is: no critical risks identified in the implementation layer.

3. Key Security Improvements

This section documents the defensive patterns implemented in LockStakingV3 that distinguish it from typical lock-staking contracts and address common vulnerability classes.

3.1 SafeERC20 Transfer Semantics

All token transfers route through an internal SafeERC20 library that tolerates non-standard ERC20 implementations (tokens that do not return a boolean). This eliminates the revert risk previously present when interacting with tokens such as USDT on Ethereum mainnet.

```
//
token.safeTransferFrom(msg.sender, address(this), amount);
uint256 received = token.balanceOf(address(this)) - balBefore;
require(received >= minDeposit, "received < minDeposit");
```

Additionally, the deposit flow measures the actual balance delta rather than trusting the input amount. This protects against fee-on-transfer tokens and other non-standard transfer semantics that would otherwise cause the contract's internal principal record to diverge from real holdings.

3.2 Internal Principal Accounting

The contract maintains two internal accumulators — totalPrincipal and totalUnpaidYield — rather than relying on token.balanceOf(address(this)) for liquidity decisions. This prevents an attacker from manipulating contract behavior by directly transferring tokens to the contract address.

```
//
function availableYieldLiquidity() public view returns (uint256) {
    uint256 bal = token.balanceOf(address(this));
    uint256 reserved = totalPrincipal + totalUnpaidYield;
    if (bal <= reserved) return 0; return bal - reserved;
}
```

3.3 Yield Shortfall Tracking

When the contract has insufficient liquidity to pay accrued yield at claim or withdraw time, the shortfall is recorded in the position's unpaidYield field (and summed into totalUnpaidYield) rather than silently discarded. Users retain an on-chain claim on owed funds and can recover them via the claimShortfall

entry point once liquidity is restored.

The withdraw function preserves the original principal field for historical record-keeping rather than zeroing it, aiding post-hoc dispute resolution.

3.4 Reentrancy Protection

A standard ReentrancyGuard pattern is used with explicit _NOT_ENTERED and _ENTERED constants. The semantics follow OpenZeppelin conventions (1 = not entered, 2 = entered), the constructor initializes correctly, and the guard is applied to every state-mutating external entry point.

3.5 Checks-Effects-Interactions Discipline

State mutations precede token transfers throughout the contract. In the withdraw function in particular, totalPrincipal, totalUnpaidYield, lastClaimTime, and principalWithdrawn are all updated before any transfer call is made. The referral close hook is invoked with its own try/catch guard to prevent a faulty registry from blocking user withdrawal.

3.6 Two-Step Ownership Transfer

Ownership transfer follows the propose-and-accept pattern. The current owner sets pendingOwner; the new owner must call acceptOwnership from the intended address. This eliminates the risk of permanently bricking the contract by transferring to a mistyped address.

```
function acceptOwnership() external {
        require(msg.sender == pendingOwner, "not pending owner");
        //
}
```

3.7 Isolated External Module Calls

All calls to the optional IReferralRegistry and IRewardsVault modules are wrapped in try/catch blocks. A malicious or buggy registry cannot DoS the deposit or withdraw flows. Additionally:

- The referral direct-bps return value is sanity-capped at MAX_REFERRAL_BPS (20%) to prevent a compromised registry from inflating referral payouts.
- Simple self-referral and two-hop cycles are detected and short-circuited.
- setRegistry and setRewardsVault both require the target to either be the zero address or contain contract code, preventing assignment to an EOA.

3.8 Rate-Limited and Lifetime-Capped Emergency Withdrawals

The emergencyWithdraw function is subject to two independent caps:

- Per-window cap: 10% of balance-at-window-start, per 24-hour window.
- Lifetime cap: 30% of a one-time snapshot taken on first invocation.

Critically, funds flow to a separate treasury address — not msg.sender — so a compromised owner key cannot unilaterally extract user funds. The lifetime cap closes the prior-version vulnerability in which an attacker could drain the contract through repeated per-window withdrawals.

3.9 Yield Accrual Cap at Maturity

The pendingYield function caps elapsed time at the lock end date. Yield does not continue accruing indefinitely after the lock matures, which both protects the protocol's liabilities and makes the yield liability deterministic.

```
//
uint256 endTime = uint256(p.startTime) + uint256(p.lockDuration);
uint256 upTo = block.timestamp < endTime ? block.timestamp : endTime;
```

3.10 Arithmetic Precision

Yield is computed with combined multiplication before division to minimize precision loss:

```
//
return (p.principal * p.aprBps * elapsed) / (365 days * BPS_DENOMINATOR);
```

For realistic inputs (principal up to 10^30, APR up to 28000, elapsed up to 10^8) the intermediate product fits comfortably within uint256.

3.11 Storage Packing

The Position struct packs startTime, lastClaimTime, lockDuration, aprBps, and principalWithdrawn into two storage slots rather than six, reducing SSTORE costs on deposit by approximately 20,000 gas per position.

4. Residual Considerations (Informational)

The following items are not security vulnerabilities but are recorded for the project team's awareness. None affect the audit result.

4.1 Economic Model

The APR schedule (50% / 120% / 280% annualized) is not supported by any on-chain revenue source encoded in the contract. Sustainability depends entirely on off-chain inflows — whether from protocol revenue, external yield strategies, or fresh deposits. The contract's internal mechanisms (principal reservation, yield shortfall tracking, emergency withdrawal caps) limit the blast radius of shortfalls but

cannot create yield where none exists.

Recommendation: the project team should document the yield source publicly and ensure inflows are sufficient to meet accrued liabilities. This is a product-governance matter outside the scope of code-level audit.

4.2 Centralization

The owner retains significant capabilities, including setting the referral registry, rewards vault, fee wallet, and treasury, and pausing deposits. While emergency withdrawals are rate-limited and lifetime-capped, and two-step ownership transfer reduces the risk of accidental loss, the owner role should ultimately be held by a multi-signature wallet (e.g., Gnosis Safe) with a time-lock for sensitive configuration changes.

4.3 Position Array Growth

The userPositions[user] array appends on every deposit and is never pruned. Users with hundreds or thousands of positions will cause the view function to return large arrays. This does not affect on-chain correctness, but off-chain indexers should paginate or filter by principalWithdrawn status.

4.4 Unused Referral Level 2

If an inviter's inviter (inv2) is address(0) or equal to the original user, the L2 reward r2 is still calculated but never delivered to a non-zero address. The current logic handles this by passing address(0) to rewardsVault.accrue; vault implementations should be aware that to2 may be zero.

5. Audit Methodology

The audit was performed through manual code review supplemented by compilation verification. Review targets included:

- Access control and privilege boundaries across all externally-callable functions
- State transitions, particularly across the deposit → claim → withdraw lifecycle
- Arithmetic operations under extreme input conditions
- Reentrancy surfaces and checks-effects-interactions compliance
- ERC20 interaction semantics including non-standard and fee-on-transfer tokens
- External module integration failure modes
- Event emission completeness and accuracy
- Storage layout and gas efficiency

The contract was compiled with Solidity 0.8.20 at optimizer runs = 200. Compilation completed with zero warnings and zero errors, producing a bytecode of 11,976 bytes (48% of the 24,576-byte EIP-170 limit).

6. Disclaimer

This audit covers the LockStakingV3 source code as provided. The audit scope does not extend to:

- The economic sustainability of the advertised APRs
- Off-chain components, front-end interfaces, or deployment infrastructure
- The trustworthiness of the owner / treasury key holders
- External contracts that may be assigned as registry or rewardsVault after deployment
- Unknown categories of vulnerabilities not enumerated in the audit checklist

Users of the deployed contract should independently verify the deployed bytecode matches audited source, confirm the owner is held by a multi-signature or governance contract, and verify any registry / vault modules assigned post-deployment are themselves audited.